

README

LLMProvider Module

`digital.vasic.llmprovider` is a generic, reusable Go module providing LLM provider abstractions and utilities. It defines the core `LLMProvider` interface and common patterns for building LLM provider implementations, including circuit breakers, health monitoring, retry logic, and lazy loading.

Features

- **LLMProvider Interface:** Unified interface for all LLM providers with `Complete`, `CompleteStream`, `HealthCheck`, `GetCapabilities`, `ValidateConfig`
- **Circuit Breaker:** Fault tolerance pattern with closed/open/half-open states to prevent cascading failures
- **Health Monitor:** Configurable health tracking with thresholds, intervals, and status transitions (healthy, degraded, unhealthy, unknown)
- **Retry Logic:** Exponential backoff with jitter, HTTP status code detection, and context cancellation support
- **Lazy Provider:** Lazy initialization pattern with deferred provider creation and optional event bus integration
- **Thread-Safe:** All components designed for concurrent use with proper synchronization
- **Minimal Dependencies:** Only depends on `digital.vasic.models` and `github.com/sirupsen/logrus`

Installation

```
go get digital.vasic/llmprovider
```

Quick Start

```
package main
```

```
import (
```

```

        "context"
        "digital.vasic.llmprovider"
        "digital.vasic.models"
        "fmt"
    )

    // Example provider implementation
    type MyProvider struct {
        llmprovider.LLMProvider
        name string
    }

    func (p *MyProvider) Complete(ctx context.Context, req
        *models.LLMRequest) (*models.LLMResponse, error) {
        return &models.LLMResponse{
            ID:         "resp_123",
            RequestID:  req.ID,
            ProviderID: p.name,
            Content:    "Response from " + p.name,
        }, nil
    }

    func (p *MyProvider) CompleteStream(ctx context.Context, req
        *models.LLMRequest) (<-chan *models.LLMResponse, error) {
        ch := make(chan *models.LLMResponse, 1)
        ch <- &models.LLMResponse{
            ID:         "resp_123",
            RequestID:  req.ID,
            ProviderID: p.name,
            Content:    "Stream response from " + p.name,
        }
        close(ch)
        return ch, nil
    }

    func (p *MyProvider) HealthCheck() error {
        return nil // Always healthy for example
    }

    func (p *MyProvider) GetCapabilities()
        *models.ProviderCapabilities {
        return &models.ProviderCapabilities{
            Name:         p.name,
            SupportsStream: true,
            SupportsTools: false,
            SupportsVision: false,
            MaxConcurrency: 10,
            MaxTokens:    4096,
        }
    }
}

```

```

func (p *MyProvider) ValidateConfig(config
    map[string]interface{}) (bool, []string) {
    return true, nil
}

func main() {
    // Create a provider
    provider := &MyProvider{name: "my-provider"}

    // Wrap with circuit breaker
    cb := llmprovider.NewDefaultCircuitBreaker("my-provider",
        provider)

    // Create a request
    req := &models.LLMRequest{
        ID:         "req_123",
        SessionID:  "sess_456",
        Prompt:     "Hello, world!",
        ModelParams: models.ModelParameters{
            Model:         "my-model",
            Temperature: 0.7,
            MaxTokens:    100,
        },
    }

    // Use the provider
    resp, err := cb.Complete(context.Background(), req)
    if err != nil {
        fmt.Printf("Error: %v\n", err)
        return
    }

    fmt.Printf("Response: %s\n", resp.Content)
}

```

Core Components

LLMProvider Interface

The foundational interface that all LLM provider implementations must satisfy:

```

type LLMProvider interface {
    Complete(ctx context.Context, req *models.LLMRequest)
        (*models.LLMResponse, error)
    CompleteStream(ctx context.Context, req *models.LLMRequest)
        (<-chan *models.LLMResponse, error)
    HealthCheck() error
    GetCapabilities() *models.ProviderCapabilities
}

```

```

    ValidateConfig(config map[string]interface{}) (bool, []string)
}

```

Circuit Breaker

Prevents cascading failures when providers are unhealthy:

- **Closed:** Normal operation, requests pass through
- **Open:** Provider is failing, requests are short-circuited with error
- **Half-Open:** Testing if provider has recovered (single request allowed)
- Configurable failure thresholds, timeouts, and reset intervals

```

cb := llmprovider.NewDefaultCircuitBreaker("provider-name",
    provider)
cb.WithFailureThreshold(5)
cb.WithOpenStateTimeout(30 * time.Second)

// Use the circuit breaker as an LLMProvider
resp, err := cb.Complete(ctx, req)

```

Health Monitor

Tracks provider health with configurable thresholds and intervals:

- **Healthy:** Provider responding normally
- **Degraded:** Performance issues but still operational
- **Unhealthy:** Provider failing consistently
- **Unknown:** Insufficient data to determine health
- Supports listeners for health status changes

```

monitor := llmprovider.NewHealthMonitor("provider-name", provider)
monitor.WithCheckInterval(10 * time.Second)
monitor.WithFailureThreshold(3)

health := monitor.GetHealthStatus()
if health == llmprovider.HealthStatusHealthy {
    // Provider is healthy
}

```

Retry Logic

Configurable retry with exponential backoff and jitter:

- Exponential backoff with configurable multiplier
- Jitter to prevent thundering herd problems
- HTTP status code detection (429, 500, 502, 503, 504)
- Context cancellation support
- Max attempts and max backoff duration

```

retry := llmprovider.NewRetryConfig()
retry.WithMaxAttempts(3)
retry.WithBackoffMultiplier(2.0)
retry.WithMaxBackoff(60 * time.Second)

err := retry.Execute(ctx, func() error {
    return provider.HealthCheck()
})

```

Lazy Provider

Lazy initialization pattern for expensive provider setup:

- Deferred provider initialization until first use
- Configurable timeout and retry attempts
- Optional event bus integration for provider lifecycle events
- Thread-safe initialization using `sync.Once`

```

factory := func() (llmprovider.LLMProvider, error) {
    return &MyProvider{name: "lazy-provider"}, nil
}

```

```

lazy := llmprovider.NewLazyProvider("lazy-provider", factory)
lazy.WithTimeout(30 * time.Second)
lazy.WithInitAttempts(3)

```

```

// Provider is initialized on first use
resp, err := lazy.Complete(ctx, req)

```

Configuration

Circuit Breaker Configuration

Parameter	De- fault	Description
FailureThreshold	5	Number of consecutive failures before opening circuit
OpenStateTimeout	30s	How long circuit stays open before moving to half-open
HalfOpenSuccessThreshold	3	Number of consecutive successes needed to close circuit
Timeout	30s	Request timeout

Health Monitor Configuration

Parameter	De-fault	Description
CheckInterval	30s	How often to perform health checks
FailureThreshold	3	Consecutive failures before marking unhealthy
SuccessThreshold	2	Consecutive successes needed to become healthy
DegradedThreshold	0.8	Performance threshold for degraded state (0-1)

Retry Configuration

Parameter	De-fault	Description
MaxAttempts	3	Maximum number of retry attempts
BackoffMultiplier	2.0	Exponential backoff multiplier
InitialBackoff	1s	Initial backoff duration
MaxBackoff	60s	Maximum backoff duration
Jitter	0.1	Jitter factor (0-1)

Thread Safety

All components are designed for concurrent use:

- `CircuitBreaker` uses `sync.RWMutex` for thread-safe state management
- `HealthMonitor` uses atomic operations for status updates
- `CircuitBreakerManager` is thread-safe for concurrent provider management
- `RetryConfig` is immutable after creation
- `LazyProvider` uses `sync.Once` for thread-safe lazy initialization

Dependencies

- `digital.vasic.models` - For `LLMRequest`, `LLMResponse`, `ProviderCapabilities` types
- `github.com/sirupsen/logrus` - For structured logging in circuit breaker
- Go standard library: `context`, `sync`, `time`, `net/http`, etc.

Development

Building and Testing

```
# Build
go build ./...

# Run all tests
go test ./...

# Run tests with race detection
go test ./... -race

# Format code
gofmt -w .

# Vet code
go vet ./...

# Check dependencies
go mod tidy
go mod verify
```

Adding New Features

1. Keep the LLMPProvider interface stable - changes break all implementations
2. Add new configuration options with sensible defaults
3. Ensure thread safety for all exported methods
4. Add comprehensive test coverage
5. Update documentation in README.md and CLAUDE.md

License

This module is part of the HelixAgent project. See root project for license details.

Contributing

See AGENTS.md for agent coordination guidelines and CLAUDE.md for AI assistant instructions.